

00 · 问题解析：这道题到底要我们交付什么？

文档定位

- **输入材料：** Atoms-Demo/Atoms-Demo.md （笔试挑战原文）、 图片和附件/87f5e26cab6a003fe069c6d18bcda3a1.png （文档分享权限设置截图）
- **本文目标：** 把"完成一个可运行的 Atoms Demo"这句模糊命题，翻译成**可验收的工程命题**，并提前暴露所有坑
- **本阶段约束：** 只做解析与拆解，**不写任何实现代码、不进入下一步**
- **不确定性声明：** 关于 Atoms 产品形态的部分来自公开资料还原（见 §4），可能与最新版本有偏差，标注为"待实测校准"

1. 任务还原：一句话说清

原文核心句：

实现一个**可运行的网页应用**，具备类似 Atoms 的能力与 UI 交互体验，即**通过智能体驱动的方式完成代码（应用）生成**，并将**生成的应用以可视化网页形式进行展示**。

翻译成一句工程命题：

用户用自然语言描述一个应用 → 多个智能体协作把它"设计并实现"出来 → 产物是一个**真的能点、能填、能存**的网页应用，并在同一界面里**实时预览** → 全过程可持久化、可迭代、可交付。

由此得到三条**不可让渡的支柱**（砍任何一条，题目就没做）：

#	支柱	含义	反例（做了也不给分）
P-1	Agent 驱动	生成过程由多个角色化的智能体协作完成，且过程 可见	一个输入框套一次 LLM 调用，无角色无过程
P-2	真应用产物	生成物是能交互、有状态、数据不丢的应用	生成一份 PRD / 一张截图 / 一段只读代码
P-3	可视化展示	生成物在同一界面内实时预览并可操作	只给下载链接 / 只能看代码不能点

2. 交付物清单（打分表就是这张表）

#	交付物	强制	判定标准	高频翻车点
1	可测试的 在线访问链接	✅ 必含	公网可访问、 非 localhost 、评审无需本地环境即可打开	交了个 <code>http://localhost:3000</code>
2	GitHub 源码链接	✅ 必含	仓库 public ，README 能让评审 5 分钟内跑起来	仓库私有 / 没有 README / 依赖缺失
3	笔试文档副本	✅ 必含	标题标注名字，文档末尾回填第 1、2 项	忘了回填链接
4	说明文档	★ 强烈建议	实现思路与关键取舍 / 完成程度（哪些做了哪些没做） / 后续扩展与优先级	只写"我做了什么"， 不写"我没什么"
5	AI coding 工具账单截图	➕ 可选	加分项	—

⚠️ **图片附件的隐藏要求：** 截图专门圈出「分享范围 → 互联网获得链接的人」，这是评审能否打开你文档的唯一开关，**必须设为公开**。这是被打回的最高频原因，且与代码质量无关。

3. 逐条需求解析（原文 → 真实意图 → 可验收标准）

原文要求	真实意图	可验收标准（评审会这么测）	常见踩坑
具备 真实交互 （而非纯静态展示）	有状态、有事件、 有反馈的完整链路	能完成一次端到端操作， 并观察到明确的状态变化	假按钮、假进度条、 点了没反应、"演示模式"
具备 数据持久化 （不限技术方案）	刷新/重开/换设备后数据仍在	刷新页面后项目、会话、产物仍在； 生成出来的应用里的数据也在	只存内存；localStorage 但刷新即丢；持久化写了却没接线
基本使用流程 （如初始化 / 注册 / 核心主流程）	原文明确点名"注册" → 需要身份与项目初始化	可注册/登录 → 可新建项目 → 进入核心生成主流程	跳过身份体系，导致"项目/ 会话归属"讲不清， 多轮迭代无处挂载
至少一个 延展或衍生能力	考察产品思维，不是一个孤立 demo	至少一条完整、可演示、 可复述的延展链路	罗列 5 个半成品，等于 0 个
实际可用的功能 ， 而不仅是概念验证（PoC）	评委会 真的 去点	主流程无 JS 报错、无死链、 无"待开发"占位页	预览 iframe 白屏/报错， 却宣称"开发已完成"
6–8 小时专注开发， 48 小时内提交	时间盒是硬约束	必须为部署 + 文档预留时间	8 小时全花在功能上，最后 1 小时无法部署 → 直接判不合格
不考察是否"用 Atoms 做 Atoms"	别把时间投在接入 Atoms 上	独立实现即为正解	花半天申请额度、对接未公开 API， 属于纯浪费

4. 参考产品 Atoms 拆解（我们要对标的体验基线）

产品定位：Atoms（原 MGX，DeepWisdom / MetaGPT 团队）是 AI 多智能体全栈产品开发平台（[atoms.dev](#)、[帮助中心](#)）。

角色分工（公开资料）：

角色	代号	职责
团队领导	Mike	统筹全局、协调各 Agent
产品经理	Emma	需求分析 → PRD
架构师	Bob	技术方案 → 架构图 / 数据库 / 接口设计
工程师	Alex	按设计生成代码
数据分析师	David	数据分析、优化决策

主管线：描述产品 → 查看计划（蓝图）→ **Agent** 构建 → 预览迭代 → 发布/导出

界面要素拆解（这些就是我们 UI 的对齐清单）：

- 1. **需求输入区：**自然语言输入 + 附件上传 + 模型选择 + 官方案例模板
- 2. **计划确认区：**先出蓝图，**用户确认**后再执行（不是黑箱直接开跑）
- 3. **Agent 时间线：**角色卡、当前动作、状态、耗时
- 4. **产物区：**PRD / 架构设计 / 数据库设计 / 接口设计 / 时序图，分 Tab 或分步呈现
- 5. **代码 / 文件树：**可查看、可导出
- 6. **实时预览：**iframe 渲染，可选中界面元素继续修改
- 7. **分享与导出：**Share → Export（下载完整工程 ZIP）；支持导入本地项目继续迭代
- 8. **项目 / 历史列表：**回合制多轮对话，支持持续迭代

体验精髓（比功能更重要）：过程可见（不是等 30 分钟黑箱）、**先计划后执行**、**可干预**（中途纠偏）、**产物可带走**（导出）。

5. 差异化机会：从 Atoms 的已知短板反推 (= 创新性得分点)

公开评测（如掘金 MGX 体验实录、腾讯云 Atoms 实践）暴露了以下短板。每一条都是我们的机会：

Atoms/MGX 已知短板	我们的做法	命中的评估维度
多智能体各干各的，Leader 只收状态不验质量	引入 Verifier/QA Agent ，产物必须过校验才算完成	工程思维 / 创新性
无测试角色，预览报错却宣称"已完成"	生成后 自动冒烟测试 （结构校验 + 渲染 + 交互断言），失败自动修复，如实上报	完成度 / 用户体验
需求漂移（输入明确需求却被擅自改动）	显式 需求契约 （必做 / 禁做 / 验收点），生成前锁定、生成后逐条核对	工程思维
只有前端 mock，没有真实后端与持久化	生成的应用 自带真实持久化 ，数据不丢	完成度
UI 无亮点	内置设计系统 / 主题令牌，产物默认有设计感	用户体验
预览反复报错、找不到模块	受限生成 （结构化规格 + 确定性渲染），稳定性提升一个数量级	完成度 / 工程思维
Token 消耗快、耗时不可控	分级模型 + 缓存 + 超时 + 无 key 也能演示的 Mock 模式	工程思维

结论：我们的"创新性"不来自更多功能，而来自 **"生成即可用 + 生成物可验证 + 过程可干预"** 这条与 Atoms 形成差异的质量主线。

6. 技术难点剖析（D1–D8）

编号	难点	本质矛盾	应对方向
D1	让 LLM 稳定产出"能跑"的应用	自由生成整个工程 = 不可控、不可验证	结构化 App Spec （页面/组件/数据模型/动作）+ 确定性渲染器；JSON Schema
D2	实时预览与沙箱	要在宿主页面里安全地跑生成物，并 捕获它的运行时错误	<code>iframe sandbox + srcdoc/blob URL + postMessage</code> 双向通道
D3	流式体验与事件协议	用户要看到"正在发生什么"	SSE（优先，简单可靠）或 WebSocket； 定义事件协议： <code>agent.started / agent.delta / artifact.created / preview.</code> 断线重连 + 幂等
D4	多智能体编排	"多智能体"很容易退化成"多次串行调用"	状态机 / 计划-执行-校验三段式；每个 Agent 的 输入输出都有 schema （产物型）
D5	持久化与并发	生成是长任务，中途刷新/关闭怎么办	项目 / 会话 / 消息 / 产物 / 版本 五类实体；生成状态可恢复；多标签页冲突处理
D6	迭代修改（二次开发）	对已生成应用继续提需求，改哪里？	需求 → 定位 App Spec 的对应片段 → 增量 patch → 版本化 → 可回滚。 这是"至少一个延展能力"的最佳候选
D7	成本 / 延迟 / 可用性	演示不能赌网络和额度	分级模型 + 结果缓存 + 超时取消 + 无 API key 的 Mock/回放模式 （保证任何环境）
D8	部署与交付	本机环境限制	已实测： <code>node v24.15.0 / npm 11.12.1 / pnpm 11.24.0 / python 3.11.7 / git v2.47.0</code> 全可用； docker、vercel CLI 不可用 → 需选择 Git 直连型托管平台并预留时间

7. 隐藏考点与红线

- 1. **在线链接必须真公网可访问。** localhost / 局域网 IP / 需要本地起服务的链接，等于没交。
- 2. **GitHub 仓库必须 public**，且 README 能让评审在 5 分钟内跑起来。
- 3. **笔试文档标题要标注名字，分享范围要"互联网获得链接的人"**（图片附件专门强调了这一点）。
- 4. **48 小时窗口 ≠ 8 小时工作量：**必须把部署与文档塞进时间盒，而不是"功能做完再说"。
- 5. **题目要求保密**，不得对外传播共享。
- 6. **"不考察用 Atoms 做 Atoms"：**时间应全部投在产品与工程本身。
- 7. **评审会真机点击：**主流程任何一处报错，都会同时扣"完成度"和"用户体验"两项分。
- 8. **文档会有专人评估，且注意文档评论：**坦诚写明"哪些没做、为什么没做、下一步优先级"，比堆砌吹嘘更契合"可交付性"维度。
- 9. **确认任务起算时间：**48 小时从何时开始计，直接决定排期是否需要压缩。

8. 评分策略：我们的分数从哪里来

题目给了 5 个评估维度。把它们翻译成"评审实际会做什么"，才知道力气该往哪使：

评估维度	评审实际在看什么	我们的对应动作	自检问题
完成度	功能完整性、稳定性、基本工程质量	P0 全绿；主流程零报错；TS 类型与运行时校验双保险；失败路径有兜底	一个陌生人从零走一遍主流程，会在哪里卡住？
工程思维	任务分解、技术选型、复杂度控制与取舍	三层适配器；可回退的 A/B 方案；P0/P1/P2 分层； 明确的"不做"清单	如果时间被砍掉一半，我先砍哪一块？为什么是它？
用户体验	交互是否清晰、流程是否顺畅、是否"可用"	三栏布局对齐 Atoms；过程可见；状态/进度明确；失败有解释；空状态有引导	第一次打开的人，知道下一步该点哪里吗？
创新性	是否有亮点设计、独特视角、可扩展潜力	质量校验自愈 + 需求契约 + 版本回滚（ 正面对冲 Atoms 的公开短板 ）	我们比 Atoms 好在哪？能用一句话说明吗？
可交付性	文档清晰度、可运行程度、整体完成质量	在线链接 + public 仓库 + README + 说明文档（含"没做什么"）+ 录屏	评审能在 5 分钟内把它跑起来吗？

分数优先级判断（这一条决定了排期）：

- 1. 门槛项 = 完成度 + 可交付性 → 不达标直接出局，再多亮点也无效。
- 2. 区分项 = 工程思维 + 用户体验 → 大多数候选人都能"跑通"，差距全在这里。
- 3. 拔高项 = 创新性 → 只有在前两项稳住时，亮点才值钱。

所以资源分配顺序是：可运行 → 可交付 → 好用 → 有想法。
反过来做（先追亮点、最后才想部署和文档）是这类限时笔试**最常见的失败模式**。

另外记住：在说明文档里如实写清"哪些没做、为什么没做"，在"可交付性"上是**加分**而不是减分——评审要看的是你的判断力，不是完美。

9. 技术选型分析（只给分析与建议，等你拍板）

数据时效：本节平台/模型信息核实于 **2026-09-29**。免费额度与限额变动极快，**落地前需以官网为准**（我会在实施时逐项复测）。

9.0 选型尺子：先定评价标准，再谈好坏

脱离约束谈技术栈是空话。本项目的真实约束如下，权重从高到低：

权重	约束	含义
★★★★★	6–8h 时间盒	任何方案，只要"搭起来"超过 1 小时就要扣分
★★★★★	必须公网可访问	部署链路必须短且一次成功，不能有"最后一步卡住"的风险
★★★★★	必须真持久化	且 跨设备可验证 更佳（评审可能换浏览器）
★★★★☆	无 LLM Key（你当前状态）	系统必须 无 key 也能完整演示
★★★★☆	无云平台账号（你当前状态）	优先"邮箱注册即可、免信用卡、免实名"的平台
★★★★☆	主流程零报错	稳定性 > 功能数（评估维度里的"完成度/用户体验"）
★★★☆☆	体现工程思维	评审想看结构、抽象与取舍，不只是"能跑"
★★☆☆☆	本机无 docker / vercel CLI	已实测，见 §10；不能用 CLI 一键部署的路径要谨慎

由这把尺子直接推出的三条结论：

- 1. "无 key 也要能演示"不是降级方案，而是 **P0 需求**（你当前就没有 key）。
- 2. 部署平台的选择会反向决定持久化方案，两者必须一起定，不能分开选。
- 3. 凡是可以后置的决策，都应该用适配器隔离掉（见 §9.10），把不确定性变成配置项。

9.1 架构形态（第一个必须定的决策）

方案	形态	优点	缺点	部署难度	时间成本	建议度
A	纯前端 SPA (React) + 浏览器持久化 (IndexedDB) + 浏览器直连 LLM	最快；零服务端；零冷启动； 静态托管随便放；无密钥泄露风险 (用户自带 key, 即 BYOK)； 调试最简单	数据只在本浏览器， 换设备即"丢失"；LLM 调用受 CORS 限制 ；"服务端工程"含量低， 工程思维得分偏弱	★ (最低)	最低	★★★
B	单仓全栈 (Next.js App Router: UI + Route Handlers 同仓) + 托管数据库 + 服务端调用 LLM	工程完整度与稳定性平衡最好； 密钥在服务端；跨设备持久化； 编排/日志/校验都做好； 部署平台零配置支持	Serverless 平台 文件系统只读 (不能用本地 SQLite 文件)； 函数有执行时长上限 (长任务需拆步或用流式)； 本地与线上有差异	★★	中	★★★★★
C	前后端分离 (Vite 前端 + 独立 Node 服务 + SQLite 文件)	最接近真实生产工程； 可长连接/WebSocket；SQLite 单文件持久化最省心； 无函数超时问题	需要 容器型平台 (且大多要信用卡或会睡眠)； 本机 无 docker ；部署链路最长， 最容易在最后 1 小时翻车	★★★★★ (最高)	最高	★★
D	纯前端 + BaaS (前端直连 Supabase/Neon)	无需自建后端即可拿到 Postgres 持久化；跨设备可验证；静态托管	业务逻辑仍在浏览器； 多引入一个账号与 SDK	★★	中低	★★★★★

关键洞察（B 方案的主要风险与解法）：

Serverless 平台的"函数执行时长上限"是 B 的最大暗礁——一次完整生成可能要 30–120 秒。**解法：把长任务拆成"由前端驱动的短步骤流"**（每个 Agent 一步、每步一次短请求，前端串起来 + SSE 流式推送）。这样：

- 每次请求都在平台超时以内 → **任何平台都能跑**；
- 进度天然可观测（正好是 Atoms 的体验精髓）；
- 中断/重试粒度更细，长任务刷新可恢复（呼应 M9）。

这个"拆步"设计让 **B 的风险从"高"降到"中"**，也是我推荐 B 而不是 C 的主要原因。

9.2 前端框架与 UI 库

选项	优点	缺点	建议
Next.js (App Router) + TS	全栈一体、部署零配置、SSE/流式原生支持、生态最全	相对重；RSC 心智负担；Cloudflare 需适配器	✅ 推荐（配 §9.1-B）
Vite + React + TS	启动极快、构建极快、心智最简单、静态部署最省事	无服务端（需配 §9.1-A/D 或另起服务）	✅ 推荐（配 §9.1-A/D）
Nuxt / SvelteKit / Remix	各自优秀、体积小	学习/踩坑成本换不来 6–8h 内的收益	❌ 不推荐
纯 HTML + 原生 JS	零构建	三栏交互 + 流式状态机手写会崩	❌ 不推荐

UI 库：

选项	优点	缺点	建议
Tailwind + shadcn/ui	视觉默认就好、可控、可复制粘贴组件、免设计	需要逐个拷组件，前期花 20–30 分钟	✅ 推荐 ("UI 无亮点"正是 Atoms 的短板，我们要在这拿分)
Ant Design	组件最全、中文场景熟悉、表单/表格开箱即用	默认观感偏"后台管理系统"，缺产品感	🔴 备选（时间极紧时用它省时间）
MUI	生态成熟	定制成本高、观感偏 Google 风	❌

9.3 后端 / 编排运行时

选项	优点	缺点	建议
Next.js Route Handlers (Node runtime)	与前端同仓同部署；SSE 原生； 无需第二个服务	受平台函数时长限制 → 用 §9.1 的"拆步"化解	✅ 推荐
Node 独立服务 (Hono / Fastify)	轻、快、可控、可长连接	多一个部署单元	🔴 备选（若选 §9.1-C）
Python (FastAPI + 多智能体生态)	多智能体/编排生态最成熟 (MetaGPT、LangGraph 都在 Python)	与前端两套语言两套部署；依赖体积大； 本机 Python 3.11 可用但部署链路显著变长	⚠️ 除非你想突出 Python 工程能力，否则不建议
纯浏览器（无后端）	零部署、零冷启动、CORS 之外无阻碍	见 §9.1-A 的缺点	🔴 备选（方案 A/D）

注：Atoms/MGX 本身是 Python（MetaGPT 系）技术栈，但**题目明确不要求复刻其技术栈**，我们的评分来自产品与工程质量，不是语言选择。**不要在 6–8h 内做双语言工程。**

9.4 生成策略（本项目最大的技术决策）

维度	① 结构化 Spec + 确定性渲染	② 自由代码生成（LLM 写 React → 沙箱构建运行）	③ 混合：Spec 为主 + 导出源码
稳定性	★★★★★（同一 Spec 渲染结果确定）	★★（易白屏/缺依赖/模块找不到）	★★★★★
产出速度	秒级	数十秒～数分钟	秒级（导出为异步）
可校验性	★★★★★（Spec 是结构化数据， 可逐条断言）	★（只能跑起来看，难自动断言）	★★★★★
可增量修改	★★★★★（定位 Spec 片段改）	★★（改坏原功能概率高）	★★★★★
表达力上限	★★★（先覆盖 表单/列表/详情/看板）	★★★★★（理论上无上限）	★★★★★
实现工作量	中（要自己写渲染器）	高（要沙箱 + 构建 + 依赖 + 错误回传）	中高
6–8h 内翻车风险	低	高	中
对"生成代码"的字面贴合	中（生成的是规格，代码是渲染产物）	高	高（可导出真实源码）
评分倾向	工程思维 / 完成度 / 稳定性占优	创新性占优，但"完成度"风险极高	折中

分析结论：

- 题目原文是"完成**代码（应用）**生成"，但括号里的"应用"才是重心，且"生成的应用以**可视化网页形式**展示"才是硬指标。
- ② 的致命问题不是"难写"，而是**它把 M6（预览）和 M8（校验自愈）的风险同时放大**——而这两块正是我们用来对冲 Atoms 短板的差异化模块。**为了"更像"而牺牲"可控"**，性价比不划算。
- ③ 是最优解：主管线走 ① 保证 6–8h 内稳，再用"导出为**真实可运行源码**"作为延展能力，同时满足"真的生成了代码"的说服力，还能把 ① 的表达力上限问题绕过（导出的代码用户可以自己改）。

我的建议：③（Spec 驱动为主 + 导出真实源码作为延展能力）。 若你更看重"生成过程的震撼感"且愿意承担风险，可选 ②，但我会把 P1/P2 全部砍掉来保它。

9.5 预览与沙箱方案

方案	隔离性	实现难度	能否跑构建	错误捕获	适用
<iframe srcdoc> + postMessage	中（sandbox 属性可控）	★（最低）	否	✅ 可注入错误钩子上报	✅ P0 推荐
iframe + blob URL	中	★★	否	✅	🔴 备选
独立子域 iframe	高	★★★ （要配第二个域）	否	✅	❌ 6–8h 内不值得
WebContainer (StackBlitz)	高	★★★★	✅ 可跑真实 npm/构建	中	🔴 仅作参考 P2 亮点探索 （真"跑代码"很有冲击力， 但工作量大）
react-live 之类	低	★	否	中	❌ 安全与表达力都受限

关键设计要求：不管选哪种，必须把预览内的运行时错误回传给宿主并显式展示。Atoms 被吐槽最多的就是“预览报错却宣称完成”——这一条是我们最好拿的“用户体验”分。

9.6 持久化方案（必须与 §9.7 部署平台一起定）

方案	类型	免费额度（2026-09 核实）	跨设备	与 Serverless 兼容	账号门槛	建议
IndexedDB / localStorage	浏览器	无限 （受浏览器限制）	❌ 仅本机本浏览器	✅ 任意静态托管	无	★★★★（方案 A 的最省路径；需配"导出/导入 JSON"补偿）
SQLite 文件	单文件	免费	✅	❌ Serverless 只读文件系统不支持	无	仅配 §9.1-C 容器方案
Cloudflare D1	SQLite@Edge	5 GB	✅	✅（Cloudflare 原生）	邮箱注册、免卡	★★★★★
Turso	托管 SQLite	9 GB / 5 亿行读	✅	✅	邮箱/GitHub、免卡	★★★★★
Supabase Postgres	托管 PG	500 MB / 2 项目	✅	✅	邮箱/GitHub、免卡	★★★★★
Vercel Postgres / Neon	托管 PG	有免费档	✅	✅	GitHub 登录	★★★★★
JSON 文件 / 内存	—	—	❌	⚠️	无	❌ 不满足"持久化"

分析结论：题目写的是“具备数据持久化（不限技术方案）”——IndexedDB 完全合规。所以：

- 若你要最低风险：IndexedDB + "导出/导入项目 JSON"，配合任何静态托管，零账号风险（除 GitHub 外）。
- 若你要更强的说服力与跨设备可验证：选 D1 / Turso / Supabase 之一，成本也只是多注册一个账号。

9.7 部署平台对比（你当前没有任何账号，这一列尤其重要）

平台	免费额度	是否睡眠/冷启动	持久磁盘	账号门槛	国内访问	适合我们的形态
Cloudflare Pages + Workers (+D1)	带宽无限；Workers 10 万请求/天；D1 5 GB	无睡眠 (冷启动毫秒级)	不可写盘， 但 D1 可持久化	邮箱注册、 免信用卡	一般 (需实测)	方案 A/D/B（Next.js 需适配器）
Vercel	100 GB 带宽/月	无睡眠，函数冷启动	只读	GitHub 登录、 免卡	一般	方案 B（Next.js 原生）
Netlify	100 GB/月；300 构建分钟	无睡眠	只读	GitHub 登录	一般	静态 / 方案 B（适配器）
GitHub Pages	免费	无	无	你本来就要 GitHub	一般	方案 A（纯静态）首选
Render	750 小时/月	⚠️ 15 分钟无流量即睡眠， 冷启约 1 分钟	免费 PG 30 天后过期	需注册	一般	⚠️ 不推荐 (评审首次打开要等 1 分钟，直接扣用户体验分)
Railway	试用 5/30天，之后 1/月额度	视配置	✅ 支持卷	需注册 (免卡试用)	一般	方案 C 备选
Fly.io	❌ 新账号已无免费档	—	✅	需信用卡	—	排除
Oracle Cloud Always Free	2 ARM OCPU / 12 GB / 200 GB， 不睡眠	无	✅	⚠️ 需信用卡 + 实名， 注册易失败	一般	方案 C 的理想归宿， 但注册成本高
Hugging Face Spaces	免费 CPU 档	闲置后休眠	可配	邮箱注册	一般	🔴 兜底选项

分析结论：

1. 排除 Render：15 分钟睡眠 + 1 分钟冷启，对一个"评审点开就要用"的作品是硬伤。
2. 排除 Fly.io：新账号无免费档且要信用卡。
3. 最优组合取决于 \$9.1：
 - 选 A（纯前端）→ GitHub Pages / Cloudflare Pages，零额外账号、零冷启动、零成本。
 - 选 B（Next.js 全栈）→ Vercel（原生零配置，最省时间）或 Cloudflare Pages + @opennextjs/cloudflare（免费额度更好，但要多一层适配风险）。
 - 选 C（容器）→ 只有 Railway/Oracle 可行，注册与调试成本会吃掉 1 小时以上，6–8h 内风险最高。
4. 务必 T+0 就打空壳部署：本机无 vercel CLI、无 docker，只能走 Git 直连的网页端流程，这条路必须先验证一遍再写业务代码。

9.8 LLM 提供方对比（你当前无 Key → 本节是重点）

核实于 2026-09（来源：[Free LLM API Tiers](#)、[Free Model Landscape 2026](#)、[KDnuggets](#)）。这些数字变动极快，仅作初筛，落地前逐一复测。

提供方	免费额度	OpenAI 兼容	需信用卡	备注
Cerebras	约 14,400 请求/天（约 100 万 token/天）	✅	❌ 免卡	请求数最宽松，适合"多 Agent 多步"调用模型
Groq	约 1,000 请求/天/模型	✅	❌ 免卡	推理最快（300+ tok/s），流式体验最好
SambaNova	请求不限（限速 10–30 RPM）	✅	❌ 免卡	有 405B 级免费模型
Google Gemini	约 1,000 请求/天（Flash-Lite）	❌ 自有 SDK	❌ 免卡	JSON 质量高；⚠️ EU/EEA/UK/CH 用户无法使用免费档；免费档数据会用于训练

提供方	免费额度	OpenAI 兼容	需信用卡	备注
OpenRouter	无余额时约 50 请求/天	✓	✗	一个 key 通 400+ 模型， 适合做统一入口 ； 免费档额度太少，仅够演示
Together / OpenAI	25/5 试用额度	✓	需绑卡	额度用尽即停
国产模型（DeepSeek / GLM / Qwen / Kimi / 豆包等）	多有小额免费额度或极低单价	多数 ✓	通常需手机号	国内直连最稳、延迟最低 ； 具体免费额度需你实测提供
本地（Ollama 等）	完全免费	✓	✗	本机配置未知（本次未检查 GPU）； 仅适合本地跑通， 线上演示不可用

从“无 key”出发的三条建议：

- 主链用免卡、OpenAI 兼容的免费档（Cerebras / Groq / SambaNova 任一）→ 零成本拿到真实 LLM 效果。
- 同时实现 Mock 提供方（预置确定性产物 + 可回放）→ 保证**任何时候、任何网络、零额度**都能完整演示。这是演示的保险丝，不是可选项。
- 所有提供方藏在同一套适配器接口后（见 §9.10）→ 你随时可以换成自己的 key，或让评审用 BYOK 粘贴自己的 key。

成本量级估算（仅供判断“要不要付费”，非精确报价）：一次完整生成约 5–8 次调用、约 15–25K 输入 + 5–10K 输出 token。

- 用免费档：**\$0**
- 用中低端付费模型（如 gpt-4o-mini 级别）：**约 ¥0.05–0.1 / 次**
- 用高端模型（Claude/GPT 旗舰级）：**约 ¥1–2 / 次** → 上百次演示约 ¥100–200

结论：**免费档 + Mock 兜底足以支撑这个 Demo**，付费不是必须项。若你后续想用自己的 key，成本也在可接受范围。

9.9 三套组合方案（供你直接点选）

	方案 A · 零依赖稳妥型	方案 B · 均衡推荐型 ★	方案 C · 工程完备型
架构	纯前端 SPA	Next.js 单仓全栈	Vite 前端 + 独立 Node 服务
前端	Vite + React + TS + Tailwind/shadcn	Next.js + TS + Tailwind/shadcn	同左
持久化	IndexedDB（+ 导出/导入 JSON）	D1 / Turso / Supabase（托管 SQLite/PG）	SQLite 文件
LLM	浏览器直连（BYOK）+ Mock	服务端调用免费档 + Mock + 可选 BYOK	服务端调用 + Mock
部署	GitHub Pages / Cloudflare Pages	Vercel （或 Cloudflare + 适配器）	Railway / Oracle（容器）
需注册账号	GitHub（本来就必需）	GitHub + Vercel + 1 个数据库	GitHub + 容器平台（多数需卡）
冷启动风险	无	低	低（但注册耗时高）
时间成本	最低（省 1–1.5h）	中	最高（多 1.5–2h）
主要风险	跨设备数据不可见；CORS 限制；工程感偏弱	平台函数时长/适配器细节	部署链路长，6–8h 内翻车概率最高
适合谁	时间极紧、追求“绝对能演示”	想要分数与风险平衡（默认推荐）	有现成云账号且时间充裕

如果让我替你决定：默认走 **方案 B**；并把 **方案 A** 作为“回退开关”——只要 T+4:00 检查点没达标，就砍掉服务端与托管数据库，退化成 A（前端代码几乎不用改，因为 LLM 与存储都在适配器后面）。**这个回退能力本身就是“复杂度控制与取舍”的展示点**，可以直接写进说明文档。

9.10 决策顺序：哪些必须现在定，哪些可以后置

决策	何时必须定	原因
架构形态（§9.1）	现在	决定后面所有代码的结构
部署平台（§9.7）	现在	与持久化强耦合；且必须先验证部署链路

决策	何时必须定	原因
持久化 (§9.6)	现在	同上
生成策略 (§9.4)	现在	决定 M5 的实现形态与工作量
LLM 具体提供方 / 模型	可后置	藏在适配器后，随时可换
UI 库细节、主题	可后置	不影响架构
导出格式 (ZIP / 单文件 HTML / React 工程)	可后置	P1 阶段再定
是否做"选中元素直接改" (P2)	可后置	有余量才做

把不确定性关进适配器——这是我对本项目最重要的一条工程建议：

LLM 适配器：MockProvider | FreeTierProvider(Cerebras/Groq/SambaNova) | ByokProvider | ServerKeyProvider
存储适配器：IndexedDBStore | D1Store | TursoStore | SupabaseStore
预览适配器：IframeSpecRenderer | (P2) WebContainerRunner

有了这三层适配器，**"有没有 key""用哪个平台""要不要换存储"都变成改一行配置**，而不是重写项目。这正是 6–8h 时间盒下最值得投入的抽象。

10. 资源现状与前置准备（开工前必须补齐）

10.1 本机能力实测结论（2026-09-29 复核）

项	状态	影响
Node.js	✅ v24.15.0	现代前端/全栈方案无阻碍
npm / pnpm	✅ 11.12.1 / 11.24.0	包管理无阻碍
Python	✅ 3.11.7	若选 Python 后端可行
Git	✅ 2.54.0.windows.1	必备，满足
Docker	❌ 未安装	排除"本地容器化验证"路径，也排除了依赖 docker 的部署演练
Vercel CLI	❌ 未安装	部署只能走网页端 Git 直连（可以，但流程要先验证）

10.2 缺口清单与补齐建议

缺口	影响	最小成本解法	预估耗时	建议时机
LLM API Key	无法验证真实生成效果	注册 免卡免费档 （Cerebras / Groq / SambaNova）拿 key；同时实现 Mock 兜底	10–20 min	开工前 （可与写代码并行）
GitHub 账号/仓库	必交源码链接	建 public 仓库（atoms-demo）	5 min	T+0
部署平台账号	必交在线链接	方案 B → Vercel（GitHub 登录）；方案 A → 无需额外账号	10 min	T+0 就要打通空壳
数据库账号（仅方案 B/C）	持久化	Cloudflare D1 / Turso / Supabase 任选，均免卡	10 min	T+0
GPU（可选）	本地跑模型	不做，用免费档云端 API	—	不需要
任务起算时间	48h 窗口的真实剩余时间	需你确认（见 02-待确认决策清单.md D7）	—	现在

10.3 建议的准备顺序（并行，总计约 30–40 分钟）

- 确认 48h 起算时间与本次可投入时长（**唯一的信息缺口，影响排期**）

2. 注册 GitHub + 建仓库（必做）
3. 注册 1 个免卡免费 LLM 档位，拿到 key（可后置到 T+1:00，但**不要晚于**，否则真实效果无法验证）
4. 选定部署平台 → **立刻部署空壳并拿到公网链接**（验证链路，最重要）
5. 选持久化托管（若方案 B）

11. 一句话结论

这不是"做一个 AI 生成代码的玩具"，而是 **"用 6–8 小时交付一个能被陌生人独立验证的、智能体驱动的应用生成产品"**。
评分的关键不是"能生成"，而是 **"生成出来的东西真的能跑、能存、能改，而且我能证明它"**。